



FACULTY OF ENGINEERING AND NATURAL SCIENCES

CAPSTONE PROGRESS REPORT

AI-BASED SMART TRAFFIC MONITORING AND MANAGEMENT SYSTEM

Mohammed Ahmed Mohammed Al-Labani, Artificial Intelligence Engineering

Abdullah Hani Abdellatif Al-Shobaki, Artificial Intelligence Engineering

Mohamed Aiman Mohamed Alkhozendar, Artificial Intelligence Engineering

Mohamed Dribika, Artificial Intelligence Engineering

Omar Kamal Burhan Jayyusi, Artificial Intelligence Engineering

Abdullah Shaher Salamoun, Artificial Intelligence Engineering

Advisors:

Prof. Murat Goksin, Artificial Intelligence Engineering

Prof. Fatih Kahraman, Artificial Intelligence Engineering

Istanbul, May 2026

STUDENT DECLARATION

By submitting this capstone project proposal, we declare that the work presented is the original work of the project team and has been prepared in accordance with the academic integrity regulations of Bahçeşehir University.

All system design decisions, methodological choices, and technical descriptions in this document were developed, reviewed, and validated by the project team. Any material, ideas, or content that are not our own have been properly acknowledged through appropriate citation.

We confirm that no unpermitted assistance was received in the design, development, or writing of this proposal, and that credit has not been falsely assigned or misrepresented among team members. We accept full responsibility for the accuracy, originality, and academic integrity of the content presented in this document.

DECLARATION OF AI USAGE

This project report was developed with the assistance of generative AI tools (e.g., ChatGPT, Copilot, Gemini, Claude, etc.). The use of AI in this work was limited to the following activities:

- Brainstorming and idea generation
- Literature search and summarization
- Grammar and language editing
- Code generation and debugging assistance
- Other (please specify): Report section drafting and content structuring based on project documentation, session logs, and experimental results produced by the team.

Validation and Authorship I certify that:

- I have personally verified all facts, citations, and data points provided by the AI.
- I have not simply copied and pasted AI outputs; all final text is my own writing or a significant rewrite of AI suggestions.
- I accept full responsibility for the integrity and accuracy of this report.

Project Team

Department 1 : Artificial Intelligence Engineering

Member 1 : Abdullah Hani Abdellatif Al-Shobaki

Member 2 : Abdullah Shaher Salamoun

Member 3 : Mohamed Aiman Mohamed Alkhozendar

Advisor : Prof. Murat Goksin

Department 2 : Artificial Intelligence Engineering

Member 1 : Mohammed Ahmed Mohammed Al-Labani

Member 2 : Mohamed Dribika

Member 3 : Omar Kamal Burhan Jayyusi

Advisor : Prof. Fatih Kahraman

ABSTRACT

This project develops an AI-based adaptive traffic management system that integrates microscopic traffic simulation, adaptive signal control, and a deep-learning computer vision pipeline within a unified real-time monitoring platform. The system targets a 3×2 arterial road network of six intersections and is composed of two principal subsystems: a SUMO-based simulation and control environment, complemented by the CARLA simulator for high-fidelity 3D visualization, and an end-to-end vehicle detection and tracking pipeline that supplies the traffic analytics required for closed-loop signal optimization.

The simulation subsystem implements an actuated signal control policy that redistributes green time based on live queue measurements, alongside a fixed-time baseline for comparison, and is exposed through a web dashboard for live visualization and policy tuning. The computer vision subsystem is built around a custom-trained YOLOv11 detector trained on a synthetic dataset generated in the CARLA simulator, paired with a multi-object tracker and an analytics pipeline that supports lane-level speed estimation and queue counting in real time. Quantitative evaluation across three demand profiles demonstrates that the actuated policy reduces network clearance time by 40–54% over the fixed-time baseline, with a chokepoint at intersections B0 and B1 identified as a target for further tuning.

The project is currently approximately 80% complete and on schedule. Remaining work for the spring term includes completing the integration of the computer vision pipeline with the control subsystem, finalizing Supabase logging, extending the comparison experiment interface, implementing red-light violation detection and highway entry counting using the existing zone framework, conducting user testing of the dashboard, and preparing the final demonstration environment.

Key Words: *Adaptive Signal Control, SUMO Microsimulation, Real-Time Traffic Dashboard, Computer Vision, Emergency Vehicle Preemption*

TABLE OF CONTENTS

1. Executive Summary	10
2. Project Objectives Recap	10
2.1 Original Objectives.....	10
2.2 Modified Objectives (if any).....	11
3. Work Completed	11
3.1 Task-by-Task Progress	11
3.2 Detailed Description of Completed Work.....	13
3.3 Individual Contributions.....	14
4. Inter-Departmental Collaboration	15
4.1 Collaboration Effectiveness	15
4.2 Integration Challenges	15
4.3 Cross-Department Deliverables.....	15
5. Updated Schedule and Gantt Chart.....	15
5.1 Schedule Comparison	15
5.2 Updated Gantt Chart	16
5.3 Remaining Work Plan.....	16
6. Preliminary Results and Findings.....	17
7. Design and Implementation Updates.....	18
7.1 Design Changes	18
7.2 Current System Architecture	19
7.3 Tools and Technologies Update	20
8. Risk Assessment Update.....	22
8.1 Realized Risks	22
8.2 New Risks	23
8.3 Revised Risk Mitigation Plan.....	24
9. Challenges and Lessons Learned	25
9.1 Technical Challenges	25
9.2 Team and Management Challenges.....	26
9.3 Lessons Learned	26
10. Budget and Resource Update.....	28
11. Advisor Feedback and Guidance	29
12. Conclusion and Next Steps	29
References	31
APPENDIX A: Meeting Minutes and Communication Logs	32
A.1 Communication Channels.....	32

A.2 Recurring Meetings.....	32
APPENDIX B: Code Repository / Technical Documentation.....	33
APPENDIX C: Additional Figures and Data.....	34

LIST OF TABLES

Table 1: Task Progress.	11
Table 2: Individual Contributions	14
Table 3: Tools and Technologies.....	20
Table 4: Realized Risks.....	22
Table 5: New Risks.....	23
Table 6: Mitigation Strategies.	24
Table 7: Budget.....	28
Table 8: Remaining Work.	29

LIST OF FIGURES

Figure 1: Gantt Chart.....	16
Figure 2: System architecture and component interactions.	19

LIST OF ABBREVIATIONS

Abbreviation	Definition
AI	Artificial Intelligence
API	Application Programming Interface
CARLA	Car Learning to Act (open-source autonomous driving simulator)
CCTV	Closed-Circuit Television
COCO	Common Objects in Context (dataset)
CPU	Central Processing Unit
CSS	Cascading Style Sheets
CV	Computer Vision
DB	Database
ERD	Entity-Relationship Diagram
FastAPI	Fast Asynchronous Python Interface (web framework)
FPS	Frames Per Second
GPU	Graphics Processing Unit
HLS	HTTP Live Streaming
HTML	HyperText Markup Language
ID	Identifier
IDF1	Identification F1 Score (tracking metric)
imgsz	Image Size
IP	Intellectual Property
JSON	JavaScript Object Notation
km/h	Kilometers per Hour
L40S	NVIDIA L40S GPU
LLM	Large Language Model
mAP	Mean Average Precision
mAP50	Mean Average Precision at IoU threshold 0.50
mAP50-95	Mean Average Precision averaged over IoU thresholds 0.50 to 0.95
MIT	Massachusetts Institute of Technology (license type)
MOTA	Multiple Object Tracking Accuracy
OOM	Out of Memory
OS	Operating System
PCB	Printed Circuit Board
PNG	Portable Network Graphics
QA	Quality Assurance

RAM	Random-Access Memory
REST	Representational State Transfer
RTX	NVIDIA Ray Tracing eXtreme (GPU series)
SORT	Simple Online and Realtime Tracking
SUMO	Simulation of Urban MObility
TLS	Traffic Light System
TraCI	Traffic Control Interface (SUMO API)
UI	User Interface
USD	United States Dollar
UUID	Universally Unique Identifier
VRAM	Video Random-Access Memory
WBS	Work Breakdown Structure
WP	Work Package
WS	WebSocket
WSL	Windows Subsystem for Linux
XML	Extensible Markup Language
YAML	YAML Ain't Markup Language
YOLO	You Only Look Once (object detection family)
YOLOE	YOLO with open-vocabulary prompting
YOLOS	YOLO based on Transformer architecture
YOLOv11	YOLO version 11
YOLOv12	YOLO version 12

1. Executive Summary

The project is an AI-driven traffic management system designed to reduce urban congestion through adaptive signal control, real-time monitoring, and computer vision-based vehicle detection. As of May 2026, the project is approximately **80% complete** across its two subsystems and is progressing on schedule. The core simulation engine, adaptive signal policy, backend API, and live dashboard have all been implemented and validated, with comparative evaluation confirming that the actuated policy reduces clearance time by 40–54% over the fixed-time baseline across three demand profiles. The computer vision pipeline covering synthetic dataset generation, model training, evaluation, and live analytics is also fully functional, with the trained YOLOv11 model exceeding initial accuracy targets and the analytics layer measuring vehicle speeds and per-lane queue lengths in real time on consumer-grade hardware.

No critical blockers exist, and the team is well-positioned to meet the spring term deadline. Outstanding work includes integrating the computer vision pipeline into the live simulation loop, finalizing the Supabase data logging pipeline, and refining the experiment comparison interface, alongside two already-designed feature deliverables (red-light violation detection and highway entry traffic counting). The principal decision points are the prioritization of computer vision integration in the next sprint and whether to pursue collision detection as a stretch goal, given its uncertain difficulty and potential impact on the remaining time budget.

2. Project Objectives Recap

The project was originally proposed as an AI-based smart traffic monitoring and management system aimed at improving traffic flow efficiency, operational awareness, and road safety in urban environments. The system was designed to be reproducible, deployable on consumer hardware, and capable of producing actionable traffic metrics — including speed, queue length, and violations — suitable for traffic engineering and smart-city applications.

2.1 Original Objectives

The original objectives of the project were as follows:

1. **Build a SUMO-based microsimulation environment modeling** a 3×2 arterial grid of six intersections, with configurable demand profiles and vehicle behaviors.
2. **Develop an adaptive traffic signal control mechanism** that adjusts signal timings dynamically based on real-time traffic density and flow conditions.
3. **Implement an intelligent exit control strategy** to regulate vehicle outflow and reduce congestion at intersections and exit points.
4. **Develop a computer vision pipeline using a fine-tuned YOLO model** to detect, track, and classify vehicles — including emergency vehicles — from simulated or real camera feeds.
5. **Detect and report** traffic violations, emergency vehicles, and road accidents to support timely traffic management decisions.
6. **Integrate the monitoring and control components** into a unified system with a centralized web dashboard for real-time visualization and analysis.
7. **Compare the adaptive policy against a fixed-time baseline** through reproducible experiments across multiple demand profiles.

8. **Log simulation data** to a persistent database (Supabase/Postgres) for post-hoc analysis and experiment archiving.
9. **Evaluate overall system performance** using simulation-based scenarios and measurable metrics such as detection accuracy, response time, and traffic flow efficiency

2.2 Modified Objectives (if any)

The core objectives of the project remain unchanged, though several additions and refinements were made during execution. Two scope additions were introduced as opportunistic extensions: an optional CARLA 3D simulation bridge for photorealistic visualization and camera-sensor testing, and an emergency vehicle priority routing module supporting manual dispatch and signal preemption, added following a mid-project review.

In parallel, two modifications were made to the computer vision approach based on early experimentation. Off-the-shelf pre-trained models (YOLOE, VideoMT) were initially planned to avoid the overhead of custom training, but testing in February and March showed they could not reliably distinguish emergency vehicles from regular traffic, prompting a pivot to training a custom YOLOv11 model. The data source was correspondingly changed from real-world footage to a CARLA-generated synthetic dataset, which afforded greater control over class balance, weather conditions, and camera placement. Collectively, these changes represent a scope expansion and methodological refinement rather than an objective substitution.

3. Work Completed

3.1 Task-by-Task Progress

Table 1: Task Progress.

Task / WP	Description	Planned Completion	Actual Status (%)	Responsible Member(s)
WP1	SUMO Simulation Environment & TraCI Integration (3×2 arterial network, demand profiles, subscription-driven state extraction)	Nov 2025	100%	Alkhozendar, Salamoun, Al-Shobaki
WP2	Signal Control Policies (actuated leftover-queue redistribution + fixed-time baseline)	Jan 2026	100%	Alkhozendar, Salamoun, Al-Shobaki
WP3	Pre-trained Model Experimentation	Feb 2026	100%	Al-Labani, Dribika, Jayyusi

	(YOLOv12, VideoMT, YOLOE)			
WP4	FastAPI Backend & WebSocket Broadcasting Layer	Feb 2026	100%	Alkhozendar, Salamoun, Al-Shobaki
WP5	Real Traffic Dataset and CARLA Dataset Generation Pipeline (capture, scenarios, batch)	Apr 2026	100%	Al-Labani, Dribika, Jayyusi
WP6	Next.js Live Monitoring Dashboard (intersection detail, metrics history, policy tuning, AI assistant)	Apr 2026	95%	Alkhozendar, Salamoun, Al-Shobaki
WP7	Emergency Vehicle Priority Routing & CARLA 3D Visualization Bridge	Apr 2026	100%	Alkhozendar, Salamoun, Al-Shobaki
WP8	YOLOv11 Model Training & Evaluation Pipeline	May 2026	100%	Al-Labani, Dribika, Jayyusi
WP9	ByteTrack Multi-Object Tracking Integration	May 2026	100%	Al-Labani, Dribika, Jayyusi
WP10	Traffic Analytics (calibration, speed, queue, violations)	Jun 2026	60%	Al-Labani, Dribika, Jayyusi
WP11	Comparative Evaluation Experiments & Supabase Data Logging	Jun 2026	70%	Alkhozendar, Salamoun, Al-Shobaki

3.2 Detailed Description of Completed Work

WP1 — SUMO Simulation Environment & TraCI Integration Built the core microsimulation environment in SUMO, modeling a 3×2 arterial road network of six intersections with configurable demand profiles (balanced, asymmetric, extreme) and realistic vehicle behaviors. Developed a TraCI-based client with subscription-driven state extraction, providing efficient per-tick access to vehicle positions, speeds, and per-lane queue measurements. This subsystem serves as the foundation for both control policies and downstream analytics.

WP2 — Signal Control Policies Implemented two signal control policies for comparative evaluation. The actuated policy dynamically redistributes green time across phases based on live per-lane queue measurements, prioritizing approaches with higher demand. A fixed-time baseline policy was also developed to serve as a reference, using static phase durations independent of traffic conditions. Both policies share a common interface to support reproducible race-mode experiments.

WP3 — Pre-trained Model Experimentation Evaluated three off-the-shelf detection options before committing to a custom-trained model. YOLOv12 with COCO pre-training detected common vehicles reliably but lacks emergency vehicle classes. Meta's VideoMT was migrated to WSL Ubuntu after Windows incompatibility but was deemed too complex to adapt for traffic detection. YOLOE, an open-vocabulary YOLO variant supporting text prompts, handled common classes but could not reliably distinguish police cars from regular cars. The team concluded that fine-grained class distinctions required a custom-trained model.

WP4 — FastAPI Backend & WebSocket Broadcasting Layer Developed a RESTful FastAPI backend exposing endpoints for simulation control, policy configuration, experiment management, and emergency vehicle dispatch. A WebSocket broadcasting layer streams per-tick simulation state to all connected dashboard clients in real time, enabling synchronized live monitoring without polling overhead. The backend acts as the integration hub between the simulation engine, dashboard, and persistence layer.

WP5 — Real Traffic Dataset and CARLA Dataset Pipeline Initially attempted to build a real-world dataset for the seven target classes using publicly sourced images annotated and augmented in Roboflow, but trained models performed poorly, prompting a pivot to a CARLA-generated synthetic dataset. Developed `setup_scenario.py` for interactive camera positioning and traffic light selection, and `capture_dataset.py` for synchronous CARLA simulation with seven-class vehicle spawning and 3D-to-2D bounding box label projection. The pipeline supports up to four simultaneous cameras, cycles through thirteen weather presets per scenario, and runs in batch mode across multiple maps. Class spawn ratios were tuned to address imbalance for rare classes.

WP6 — Next.js Live Monitoring Dashboard Built a Next.js-based live monitoring dashboard providing real-time visualization of the simulation state, including network-wide intersection views, signal phase countdowns, and vehicle tracking. The dashboard supports intersection-level detail panels, metrics history charts, manual policy tuning controls with variant saving, emergency vehicle injection, and a Groq-powered AI assistant for interactive querying. The interface communicates with the backend via REST and consumes the WebSocket stream for live updates.

WP7 — Emergency Vehicle Priority Routing & CARLA 3D Visualization Bridge Added two demonstrable extensions to the core system. The emergency vehicle priority routing module supports manual dispatch through the dashboard and applies signal preemption logic through the simulation tick loop to clear the vehicle's path. A CARLA 3D simulation bridge was also integrated to provide photorealistic visualization of the SUMO scenario, enabling camera-sensor testing for the computer vision pipeline. The CARLA bridge is treated as optional, with the system gracefully falling back to SUMO if unavailable.

WP8 — YOLOv11 Model Training & Evaluation Pipeline Two preliminary YOLOv11m models trained on the real-data dataset reached test mAP50 of 0.931 but failed when deployed on live municipal traffic feeds due to a visual-domain mismatch with the scraped training imagery. This confirmed the pivot to CARLA-generated synthetic data. The final YOLOv11m model was trained on a rented NVIDIA L40S cloud GPU via vast.ai over 39 epochs in approximately three hours, reaching mAP50 = 0.949, mAP50-95 = 0.865, precision = 0.97, and recall = 0.90. A full evaluation pipeline was also developed, including scripts for model evaluation, model comparison, dataset analysis, and report generation.

WP9 — ByteTrack Multi-Object Tracking Integration Integrated Ultralytics' built-in ByteTrack tracker on top of the YOLOv11m detector to provide ID-consistent multi-object tracking required by the downstream analytics layer. ByteTrack was selected for being appearance-free, using two-stage association to recover partially-occluded vehicles, and shipping natively within Ultralytics. Configuration was tuned for traffic footage with a 30-frame track buffer, a 0.8 match threshold, and class-conditional association to prevent silent class switches. Validation on a 4,979-frame held-out CARLA recording yielded MOTA = 0.78, IDF1 = 0.88, and 47 ID switches concentrated at expected failure modes (long occlusions, re-entries).

WP10 — Traffic Analytics (Calibration, Speed, Queue, Violations) Built an analytics layer that converts pixel detections into real-world metrics. Calibration uses 4-point homography, the industry-standard approach for retrofit traffic cameras, and is automatic for CARLA recordings. Speed is computed per tracked vehicle by applying the homography to a reference point on the bounding box, with a 5-frame rolling average for smoothing. Per-lane queue counts are derived from interactively defined lane polygons with configurable thresholds (default: speed below 5 km/h for 2 seconds). The pipeline operates at 28 FPS on consumer GPU hardware. Violation detection (red-light and highway entry counting) is designed and pending implementation.

WP11 — Comparative Evaluation Experiments & Supabase Data Logging Designed and executed a reproducible race-mode experiment framework comparing the actuated policy against the fixed-time baseline across three demand profiles, yielding six total experimental runs. Results demonstrate that the actuated policy reduces network clearance time by 40–54% over the baseline across all profiles, with a chokepoint identified at intersections B0 and B1. Initial Supabase database integration is in place for persistent logging of simulation runs and metrics, with the full logging pipeline still being finalized for archival of post-hoc experiment data.

3.3 Individual Contributions

Table 2: Individual Contributions

Team Member	Department	Tasks / Contributions	Effort (%)
Alkhozendar	AI Engineering	WP1, WP2, WP4, WP6, WP7, WP11	100
Salamoun	AI Engineering	WP1, WP4, WP6, WP11	99.9
Al-Shobaki	AI Engineering	WP4, WP6, WP7, WP11	100
Omar Jayyusi	AI Engineering	WP3, WP5, WP8, WP9, WP10	100

Mohamed Dribika	AI Engineering	WP3, WP8, WP9, WP10	100
Mohammed Al-Labani	AI Engineering	WP3, WP8, WP9, WP10	100

4. Inter-Departmental Collaboration

The project is executed jointly by two sub-teams under the AI Engineering department: the simulation and control team, responsible for the SUMO environment, signal policies, backend, and dashboard; and the computer vision team, responsible for dataset generation, model training, tracking, and analytics. Although both teams sit within the same department, the work is functionally divided along clear sub-domain boundaries, requiring deliberate coordination throughout the project.

4.1 Collaboration Effectiveness

Weekly synchronization meetings are held to align on progress, surface blockers, and coordinate interface decisions between the perception layer and the simulation backend, while day-to-day communication is handled asynchronously through a shared messaging workspace. Code and documentation are maintained in shared Git repositories using feature branches and pull requests, with larger design decisions — such as the data contract between CV outputs and the control loop — documented in shared notes.

4.2 Integration Challenges

The most significant challenge has been aligning the computer vision pipeline with the live simulation loop, since the CV subsystem was developed against recorded CARLA footage while the control policy was developed against direct TraCI state queries; bridging the two has required defining a shared schema for per-lane analytics and reconciling frame rate against the simulation tick rate. A secondary challenge has been the dual use of CARLA — as a 3D visualization bridge for the simulation team and as the synthetic dataset source for the CV team — which required coordination on maps, camera conventions, and scenarios to avoid duplicated effort. Both are being resolved through the CV integration work scheduled for the next sprint.

4.3 Cross-Department Deliverables

The CARLA scenario configuration was co-designed to serve both photorealistic visualization and synthetic dataset capture and is currently complete. The end-to-end integration of computer vision analytics into the live control loop, enabling detector-derived queue measurements to feed the actuated policy, is the principal pending cross-team deliverable, targeted for early spring term. The unified dashboard is partially complete, with simulation-side panels in place and live CV outputs pending integration. Comparative evaluation experiments are likewise partially complete, with SUMO-only runs executed and CV-augmented runs planned for the final demonstration phase.

5. Updated Schedule and Gantt Chart

5.1 Schedule Comparison

All tasks are being completed by their scheduled time.

5.2 Updated Gantt Chart



Figure 1: Gantt Chart.

5.3 Remaining Work Plan

The following milestones remain before the end of the spring term, organized by target completion week:

1. **CV Pipeline Integration** (*Target: end of Week 11*) — Wire the YOLOv11 detection output into the simulation tick loop so that vehicle detection feeds live into `extract_tick_data()`. Validate detection accuracy against TraCI ground truth.

2. **Supabase Logging Completion** (*Target: end of Week 11*) — Finalize tick-level data writes to Supabase Postgres, implementing batched asynchronous inserts to avoid blocking the tick loop, and verify schema alignment with the TickData model.
3. **Forbidden Line Definition** (*Target: end of Week 11*) — Extend setup_analytics.py with a 2-point line picker to support the violation detection module.
4. **Red-Light Violation Detection** (*Target: mid Week 12*) — Detect when a tracked vehicle crosses a forbidden line during a red light phase, leveraging the existing zone-and-light-state infrastructure.
5. **Highway Entry Counting** (*Target: mid Week 12*) — Count vehicles entering tagged entry zones grouped by the current signal light state.
6. **Experiment Comparison UI Polish** (*Target: mid Week 12*) — Complete the frontend experiment results page with side-by-side metric comparison charts and race outcome summaries.
7. **User Testing and Dashboard QA** (*Target: end of Week 12*) — Conduct internal user testing of the full dashboard flow, covering simulation start, live monitoring, policy tuning, emergency dispatch, and experiment comparison.
8. **Final Demo Environment Setup** (*Target: start of Week 13*) — Package the system for demonstration by configuring stable launch scripts, preparing sample demand profiles, and rehearsing the demo walkthrough.
9. **Final Report Submission** (*Target: end of Week 13*) — Complete and submit the final project report documenting the full system design, implementation, evaluation results, and conclusions.

6. Preliminary Results and Findings

The project has progressed from design and prototyping into a validated state, with measurable results available from both subsystems.

On the adaptive signal control side, a race-mode experiment framework was used to benchmark the actuated policy against a fixed-time baseline across three demand profiles (balanced, asymmetric, and extreme). Across all profiles, the actuated policy reduced network clearance time by 40–54% relative to the baseline, confirming that queue-driven green-time redistribution produces substantial throughput gains. A recurring chokepoint at intersections B0 and B1 was identified as a target for further tuning. The Next.js dashboard was validated end-to-end against the running simulation, rendering the full 3×2 network with live signal phases and per-intersection metrics in real time, and supporting policy tuning, emergency vehicle dispatch, and interactive querying through the embedded AI assistant.

On the computer vision side, the trained YOLOv11m model achieved precision = 0.952, recall = 0.823, mAP50 = 0.949, and mAP50-95 = 0.865 on a held-out 4,979-frame test recording, with strong per-class F1 scores across all seven vehicle types. ByteTrack tracking yielded MOTA = 0.78 and IDF1 = 0.88 with only 47 ID switches across 47,014 detections, and combined inference runs at 27.7 FPS on consumer-grade hardware — exceeding the 20 FPS real-time target. The analytics layer was validated on a CARLA scenario, with per-vehicle speeds and per-lane queue counts updating correctly in real time. Confusion matrix analysis indicated that the dominant failure mode is missed detections rather than class confusion.

These results confirm that the project's core objectives have been met or exceeded: the control subsystem delivers measurable improvements in network performance, the perception subsystem provides reliable real-time detection including fine-grained emergency vehicle classification, and the dashboard unifies both under a single interface. The remaining work builds directly on this validated foundation rather than introducing new technical risk.

7. Design and Implementation Updates

7.1 Design Changes

Several design changes were made during implementation, driven by practical constraints, emerging requirements, and findings from early experimentation. They are presented below grouped by subsystem.

On the simulation and control side:

1. **CARLA added as an optional second simulation engine** — The original design specified SUMO as the sole simulation backend. CARLA was subsequently added as an alternative engine to support photorealistic camera-sensor simulation and computer vision validation, since SUMO's abstract vehicle representations were insufficient for real-world generalization of the CV model. Rather than replacing SUMO, CARLA was integrated as a hot-swappable mode behind the same WebSocket interface, allowing the dashboard to remain engine-agnostic.
2. **Emergency vehicle preemption module added** — Not present in the original proposal, this module was added mid-project as a high-value demonstrable feature. It extends the signal control layer with a preemption path that overrides the actuated policy when an emergency vehicle is dispatched, clearing a green corridor toward the target intersection.
3. **Policy made fully parameterizable and persistent** — The original actuated policy used hardcoded parameters. It was redesigned to expose eight tunable fields (ActuatedPolicyParams) through a frontend panel, with named variants saved to disk under artifacts/policy_variants/. This change was necessary to support iterative tuning and fair experimental comparison across configurations.
4. **Route file regenerated on every non-race start** — The route file was originally a static artifact. The design was changed so that SimulationManager.start() regenerates arterial.rou.xml from the chosen demand profile and vehicle count before each run, allowing demand profile selection from the dashboard to take effect.
5. **Groq AI assistant added to dashboard** — Not in the original design. A floating chatbot powered by the Groq API was embedded in the dashboard to allow operators to query simulation state in natural language, providing a low-effort, high-demonstration-value addition.

On the computer vision side:

6. **Pre-trained model replaced with a custom-trained model** — The original concept relied on off-the-shelf models with text prompts (YOLOE). Testing in March 2026 revealed that prompt-based detection could not reliably distinguish police cars from regular cars, prompting a change in architecture to train a custom YOLOv11 model on labeled data.
7. **Real-world dataset replaced with CARLA synthetic data** — The team initially built a real-world training dataset from public sources (Kaggle and scraped images) for the seven target classes, annotated and augmented through Roboflow. Although validation metrics on the

cleaner of two trained models were strong (test mAP50 = 0.93), deployment on live municipal traffic feeds revealed substantial failures due to visual-domain mismatch and severe class imbalance in the scraped imagery. The team pivoted to CARLA-generated synthetic data, where class distribution, lighting, and camera placement are all controllable.

8. **Local training replaced with cloud training** — Local training on the RTX 4060 Laptop became infeasible as the dataset grew. Training was migrated to a rented cloud GPU (vast.ai NVIDIA L40S 48GB), completing in approximately three hours at a cost of around \$2.5 USD, compared to an estimated 25+ hours locally.

7.2 Current System Architecture

The current system is organized into four cooperating layers — the simulation engines, the FastAPI backend, the web dashboard, and the computer vision pipeline — connected through a real-time data flow that is illustrated in Figure 1.

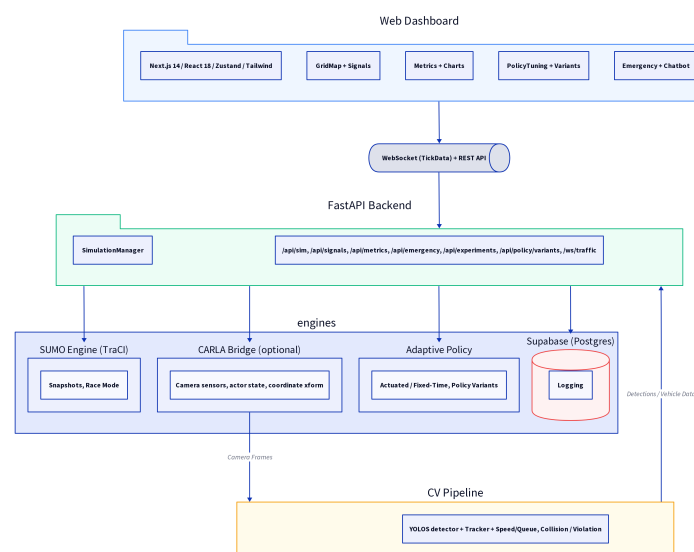


Figure 2: System architecture and component interactions.

At the foundation, the simulation engine (SUMO or CARLA, hot-swappable behind a common interface) produces raw vehicle and signal state on every tick. The FastAPI backend extracts this into a structured TickData snapshot, applies the active signal control policy, pushes the resulting commands back to the simulation, logs the tick to Supabase, and broadcasts the snapshot over WebSocket. The Next.js dashboard consumes TickData frames through a useWebSocket hook, updates the Zustand store, and renders the live map, intersection panels, metrics history, and policy tuning interface.

The computer vision pipeline runs in parallel and is implemented in Python under `capstone_sim/scripts/`, organized around four stages. The *capture* stage (`capture_dataset.py`, `record_test.py`) connects to a CARLA simulator instance, spawns vehicles from configurable scenario YAML files, and outputs YOLO-format datasets or sequential PNGs with ground-truth labels. The *training* stage (`train.py`) wraps the Ultralytics YOLO API for training YOLOv11 on the captured dataset. The *evaluation* stage (`evaluate_model.py`) runs the trained model with ByteTrack and computes both detection metrics (precision, recall, mAP, confusion matrix) and tracking metrics (MOTA, IDF1, ID switches). The *analytics* stage performs camera calibration via 4-point homography and interactive lane definition (`setup_analytics.py`), and runs the analytics pipeline on either recorded video (`traffic_analytics.py`) or a live CARLA simulation (`live_analytics.py`). Shared modules in `scripts/` provide constants, bounding-box projection, CARLA helpers, and visualization tools. Scenario

definitions are stored as YAML under `capstone_sim/configs/`, and per-scenario analytics configurations (calibration matrix and lane polygons) are saved to `capstone_sim/analytics_configs/`.

In the integrated system, the CV pipeline operates as an independent producer alongside the SUMO/CARLA state stream, generating its own vehicle detections, tracking IDs, speed estimates, queue counts, and violation flags. These outputs are intended to feed the same TickData channel during the upcoming integration milestone, allowing the actuated control policy to operate on perception-derived measurements in addition to ground-truth simulation state.

7.3 Tools and Technologies Update

The complete stack of tools and technologies used across both subsystems is summarized in the table below. The *Status* column indicates whether each item was carried over from the original proposal (✅), added during the project (➕), or modified relative to the original plan (⚙️).

Table 3: Tools and Technologies.

Category	Tool / Technology	Status
Simulation & Control		
Traffic simulation	SUMO 1.26.0 (TraCI Python API)	✅ As proposed
3D simulation	CARLA 0.9.16 (optional visualization bridge)	➕ Added
Signal control	Actuated (leftover-queue) + fixed-time policies	✅ As proposed
Policy persistence	Filesystem (artifacts/policy_variants/)	➕ Added
Backend & Data		
Backend framework	FastAPI + Uvicorn + WebSocket	✅ As proposed
Database	Supabase (Postgres)	✅ As proposed
Backend language	Python 3.9+	✅ As proposed
Frontend		
Frontend framework	Next.js 14 + React 18 + Tailwind CSS	✅ As proposed
State management	Zustand	✅ As proposed
Frontend language	TypeScript	✅ As proposed
AI assistant	Groq API (LLM chatbot in dashboard)	➕ Added
Computer Vision		

Category	Tool / Technology	Status
Detection framework	Ultralytics YOLO 8.4 (YOLOv11 architecture)	⚙️ Changed from YOLOs
Multi-object tracking	ByteTrack (integrated via Ultralytics)	⚙️ Changed from SORT/DeepSORT
Deep learning framework	PyTorch 2.x (with CUDA)	✅ As proposed
Image processing	OpenCV (frame I/O, drawing, homography, video output)	✅ As proposed
Numerical computing	NumPy, Matplotlib, PyYAML	✅ As proposed
Dataset annotation	Roboflow (annotation, augmentation, YOLO export)	+ Added
Live-stream ingest	yt-dlp + ffmpeg (HLS feeds, real-data attempt)	+ Added
Analytics pipeline	Custom homography-based ground-plane calibration	+ Added
Compute & Infrastructure		
Cloud GPU (final training)	vast.ai (NVIDIA L40S 48GB)	+ Added
Cloud GPU (preliminary)	Google Colab (NVIDIA A100)	+ Added
Local hardware	NVIDIA RTX 4060 Laptop GPU	✅ As proposed
Development & Tooling		
Version control	Git / GitHub	✅ As proposed
Package management	pip (backend) + npm (frontend)	✅ As proposed
Linting	Ruff (line-length 100)	✅ As proposed
OS / environment	Windows 11 + WSL Ubuntu, Python 3.11 (conda)	✅ As proposed
Standards		

Category	Tool / Technology	Status
Detection labels	YOLO label format	✓ As proposed
Tracking labels	Extended YOLO format (adds actor_id column)	+ Added
Analytics output	Ground-plane coordinate system (meters)	+ Added

The principal additions relative to the original proposal are the optional CARLA bridge, the Groq AI assistant, the policy persistence layer, the cloud GPU infrastructure (vast.ai and Google Colab) needed for the larger training runs, the Roboflow-based dataset construction pipeline used during the real-data attempt, the ByteTrack tracker integrated when multi-object tracking moved into scope, and the homography-based analytics pipeline. The principal modifications are the change from a YOLOs detector to a custom-trained YOLOv11 model, and the replacement of SORT/DeepSORT with ByteTrack as the tracking algorithm.

8. Risk Assessment Update

8.1 Realized Risks

Several risks identified at proposal time, along with new risks that emerged during implementation, materialized over the course of the project. Each was mitigated without lasting impact on the project schedule. The realized risks across both subsystems are summarized in the table below.

Table 4: Realized Risks.

Risk	Impact	Mitigation Taken
Insufficient real-world traffic video data — access to municipal CCTV footage was not granted by the relevant authorities, and publicly available data was sparse and of low quality.	High — invalidated the original plan to train on real-world imagery.	Switched to a CARLA-generated synthetic dataset, which provided full control over class distribution, weather, and camera placement.
Insufficient model accuracy on rare classes — initial training showed police_car produced 608 false positives with zero ground-truth police cars in the test scene, and bus recall was only 7.9%.	Medium — would have undermined fine-grained classification, particularly for emergency vehicles.	Rebalanced spawn ratios (doubling police_car and bus, increasing bike), regenerated the dataset, and retrained. Bus recall improved to 96% and all underperforming classes recovered.
Hardware limitations for training — local training on the RTX 4060 Laptop became infeasible as the dataset grew.	Medium — would have blocked the final training run.	Migrated training to a rented cloud GPU (vast.ai NVIDIA L40S 48GB), reducing training time from an estimated 25+ hours to approximately 3 hours.

Risk	Impact	Mitigation Taken
CARLA GPU instability on Town03 — Town03 crashed during GPU rendering at the CARLA integration stage.	Medium — delayed CARLA camera validation by approximately one session.	Pivoted to Town10HD, which is stable and provides equivalent sensor coverage. No functional loss.
TraCI race-mode second-run failure — the second comparison run crashed due to an incorrect socket teardown order in the simulation manager.	Medium — blocked the automated six-race evaluation temporarily.	Enforced a strict teardown sequence in SimulationManager. All six races now complete reliably.
TypeScript/Python type drift — frontend types fell out of sync with backend Pydantic models as new fields were added.	Low — caused minor runtime errors caught during development.	Established a co-commit convention requiring TypeScript types to be updated in the same commit as their Pydantic counterparts.

8.2 New Risks

Several new risks have emerged during implementation that were not anticipated at the proposal stage. They are summarized in the table below.

Table 5: New Risks.

Risk	Likelihood	Impact	Severity
CV pipeline integration delay — the YOLOv11 model is ready but wiring it into the live simulation tick loop may surface performance or latency issues not yet observed.	Medium	Medium	Medium
Supabase write performance under tick load — synchronous database writes during each simulation tick could slow the loop below the real-time threshold.	Medium	High	High
B0/B1 intersection chokepoint — the structural network geometry at intersections B0 and B1 creates persistent queue buildup that the actuated policy cannot fully resolve through timing alone.	High	Medium	Medium
Demo environment reproducibility — running the full stack (SUMO, FastAPI, Next.js, Supabase) on demonstration hardware may produce configuration drift or port conflicts.	Low	Medium	Medium
Single-camera occlusion bias — ByteTrack loses identity through long occlusions (e.g., bikes hidden behind larger vehicles), reducing tracking metrics for affected classes.	High	Medium	Medium

Risk	Likelihood	Impact	Severity
Out-of-memory failures during model training — large dataset and batch sizes can exhaust GPU memory during training runs.	Medium	Medium	Medium
CARLA simulation instability — vehicles occasionally become stuck in the simulation, affecting both dataset capture and live runs.	Medium	Medium	Medium

8.3 Revised Risk Mitigation Plan

The mitigation strategies adopted in response to the realized and newly identified risks are summarized in the table below.

Table 6: Mitigation Strategies.

Risk	Mitigation Strategy
CV pipeline integration delay	Timebox integration to Week 10. If latency issues emerge, run the CV pipeline in a separate asynchronous process decoupled from the tick loop, consuming snapshots from a shared queue.
Supabase write performance	Switch to batched asynchronous writes — buffer ten ticks and flush asynchronously using Supabase's bulk insert API. A database write timeout is configured so that a slow write never blocks the simulation.
B0/B1 intersection chokepoint	Expose per-intersection maximum green time as a tunable parameter in the policy panel, allowing manual green-time bias for B0 and B1 as a near-term workaround. The structural limitation will also be documented as a known network geometry constraint in the final report.
Demo environment reproducibility	Provide a single-command launch script (scripts/start_demo.sh) that starts all services in the correct order with health checks. Rehearse the demonstration at least twice on the final hardware before the presentation.
Class imbalance in training data	analyze_dataset.py runs automatically after each capture and flags imbalances exceeding a 5× max/min ratio, enabling early correction before training.
Out-of-memory failures during training	train.py exposes a --cache flag (true / false / disk) so that RAM caching is only enabled when the dataset fits in memory, with disk caching as a fallback for larger datasets.
CARLA simulation instability (stuck vehicles)	analytics_config.yaml supports stuck_check_interval and stuck_threshold_m as tunable parameters, with conservative defaults (30-second interval, 1.0 m threshold) to preserve normal queue-formation behavior at red lights.
Single-camera occlusion bias	Documented as a fundamental limitation of single-camera detection. Partially mitigated through adjustments to camera positions; full mitigation would require multi-camera fusion, which is out of scope for this project.

Risk	Mitigation Strategy
Hardware limitations for training	Training migrated to cloud GPU rental (vast.ai NVIDIA L40S 48GB) at approximately \$0.50/hour, eliminating local hardware as a bottleneck for the remainder of the project.

9. Challenges and Lessons Learned

9.1 Technical Challenges

A number of substantive technical challenges were encountered and resolved during implementation. They are presented below grouped by subsystem.

On the simulation and control side:

1. **CARLA-SUMO coordinate frame mismatch** — SUMO and CARLA use different coordinate origins, axis directions, and rotation conventions, causing vehicles placed in SUMO to appear at completely wrong positions in the CARLA visualization. The correct transformation ($\text{carla_x} = \text{sumo_x} - 109.34$, $\text{carla_y} = 135.96 - \text{sumo_y}$, $\text{carla_yaw} = \text{sumo_yaw} - 90^\circ$) was derived empirically by cross-referencing known landmark positions in both environments. The transformation is now parameterized in the shared configuration and applied within the CARLA bridge before any coordinate is used.
2. **TraCI socket lifecycle in race mode** — The automated six-race evaluation consistently crashed on the second run because the TraCI connection was not fully closed before the next simulation process started, leaving orphaned socket handles. The issue was resolved by enforcing a strict teardown sequence in SimulationManager, ensuring that TraCI disconnects, the SUMO subprocess terminates, and all file handles close before the next run begins.
3. **Real-time WebSocket performance under high vehicle counts** — At 7,000+ vehicles, tick-loop broadcast latency became noticeable in the dashboard. Reducing the default vehicle count to 5,500 and optimizing the snapshot extraction path — using the lighter `extract_race_tick()` for race mode — brought latency back within acceptable bounds.
4. **CARLA GPU instability on Town03** — The Town03 map caused GPU memory crashes during camera sensor rendering. The team migrated to Town10HD, which is lighter and fully stable, with no loss of functionality.

On the computer vision side:

5. **CARLA synchronous-mode timing** — In synchronous mode, CARLA does not update actor transforms until `world.tick()` is called. An early bug resulted in `camera_location` reading as (0, 0, 0), causing distance filters to reject all vehicles. The issue was resolved by inserting an explicit `world.tick()` before reading the camera location.
6. **Edge-of-screen detection loss** — A `MIN_VISIBILITY` threshold of 0.4 dropped labels for vehicles more than 60% off-screen during dataset generation, meaning the model never learned to detect partially visible vehicles. Lowering `MIN_VISIBILITY` to 0.15 corrected this gap in training coverage.

7. **Rotation matrix conventions** — A manual implementation of CARLA's pitch/yaw/roll to world-rotation conversion produced rays that did not intersect the ground plane, because CARLA uses positive pitch = nose up, the opposite of the standard convention. The issue was resolved by using CARLA's built-in `carla.Transform.get_forward_vector()` and related methods directly.
8. **VRAM out-of-memory during training** — Training yolo11m at batch size 96 crashed at epoch 17 due to GPU memory exhaustion. Training was resumed with batch size 64 and `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True`, which completed successfully.
9. **Detection coverage versus class confusion** — Initial confusion matrix analysis showed that errors were dominated by missed detections rather than misclassifications, indicating that detection coverage was the bottleneck rather than class separability. The dataset was expanded with additional examples of under-represented classes and spawn ratios were tuned accordingly, improving recall across affected categories.

9.2 Team and Management Challenges

Team and management challenges were limited and largely confined to the boundary between the two sub-teams, where shared interfaces required active coordination.

Within the computer vision sub-team, no significant challenges were encountered. Work was divided early along clear ownership lines — dataset pipeline, model training, tracking and analytics, and evaluation — which kept responsibilities unambiguous and avoided overlap. Version control via Git proceeded without notable merge conflicts, and coordination around exam periods was handled by adjusting milestone deadlines in advance rather than reactively.

Within the simulation and control sub-team, and at the interface between sub-teams, three challenges did arise and were resolved during execution:

1. **Shared type system maintenance** — With the backend (Python/Pydantic) and frontend (TypeScript) maintained by different members, model fields fell out of sync multiple times as new features were added, producing hard-to-diagnose runtime errors. A co-commit convention was established: any Pydantic model change must include the corresponding TypeScript update in the same pull request.
2. **Asynchronous integration** — Sub-teams worked on their components independently, which occasionally produced integration surprises when components were merged — for example, the frontend assuming a WebSocket field that the backend had not yet added. The `docs/api_spec.md` file was established as a shared contract, with any API or WebSocket schema change required to be reflected there before implementation begins.
3. **Scope creep management** — Several features (the CARLA bridge, emergency routing, Groq chatbot, and policy variants) were added beyond the original proposal. While all proved valuable, they introduced unplanned work mid-sprint. A lightweight feature-flagging approach was adopted, with new features implemented as optional or additive extensions rather than modifications to existing behavior, reducing the risk of regressions.

Overall, collaboration was smooth across both sub-teams throughout the project lifecycle, and no challenges escalated to the point of affecting the schedule or scope of deliverables.

9.3 Lessons Learned

A number of lessons were drawn from the project's experience to date, spanning both engineering practice and the specific technical decisions made by each sub-team.

On the engineering and team-process side:

1. **Define integration contracts before implementation.** The TypeScript/Pydantic drift problem would have been avoided entirely if the shared type contract had been locked before both sides started building. Shared interfaces should be the first artifact produced, not a retroactive one.
2. **Reproducibility should be designed in from the start.** The need for reproducible, fair policy comparison was foreseeable from day one, but building race mode later required retrofitting the simulation manager and configuration system. Starting with reproducibility as a first-class design requirement would have saved significant rework.
3. **Test the full stack together early.** Integration issues such as CARLA coordinate mismatches, TraCI teardown bugs, and WebSocket field drift only surfaced when components were combined. Weekly end-to-end integration tests — even informal ones — would have surfaced these earlier and reduced debugging time.
4. **Optional dependencies should degrade gracefully.** The CARLA bridge's no-op behavior, reporting `connected: false` when CARLA is unavailable, is the right pattern. Every external dependency should degrade gracefully rather than crash the system, so that development and testing do not require every dependency to be running simultaneously.

On the modeling and data side:

5. **Pre-trained models answer "is there a vehicle?" but not "what kind of vehicle?"** Fine-grained classification — especially between visually similar emergency and non-emergency vehicles — requires training data tailored to the target classes. Off-the-shelf models cannot substitute for a properly labeled dataset when subclass distinctions matter.
6. **Class imbalance is the single biggest factor in minority-class performance.** Boosting spawn ratios for rare classes was more impactful than any model architecture change, and validated the value of investing in data quality before model tuning.
7. **Synthetic data is powerful but does not always transfer cleanly.** CARLA-generated data enabled the collection of otherwise rare scenarios and controlled class balance, but real-world deployment can still surface domain-gap issues that synthetic-only training does not anticipate.
8. **Cloud GPU rental is cheaper than local training.** At \$1–2 per training run, cloud GPU is significantly more cost-effective than the time cost of local training on consumer hardware, and removes a major bottleneck from the iteration cycle.
9. **Automated dataset validation catches problems before they consume training time.** Running an analyzer immediately after each capture surfaces class imbalances and labeling issues before they propagate into hours of wasted training.
10. **Industry-standard calibration techniques are sufficient for this class of system.** A 4-point homography approach is well-suited to the project's needs and avoids the requirement for specialty hardware, making the analytics pipeline straightforward to retrofit to real-world cameras.

10. Budget and Resource Update

The project has been developed almost entirely using open-source and free-tier tools, resulting in minimal direct monetary expenditure against the project budget. The full set of resources in use across both subsystems is summarized in the table below.

Table 7: Budget.

Resource	Tool / Service	Cost	Status
Simulation engine	SUMO 1.26.0 (open source, Eclipse Public License)	Free	In use
3D simulation	CARLA Simulator (open source, MIT)	Free	Optional
Backend framework	FastAPI + Uvicorn (open source, MIT)	Free	In use
Frontend framework	Next.js 14 + React 18 (open source, MIT)	Free	In use
Database	Supabase (free tier — up to 500 MB, 2 projects)	Free	In use
AI assistant API	Groq API (free tier for development)	Free	In use
CV detection framework	Ultralytics YOLO 8.4 / YOLOv11 (open source)	Free	In use
Multi-object tracking	ByteTrack (open source, integrated via Ultralytics)	Free	In use
Preliminary CV training	Google Colab (NVIDIA A100)	Covered by existing subscription	Used
Final CV training	vast.ai cloud GPU (NVIDIA L40S 48GB)	~\$2.50 USD	Used
Development support	Claude Max plan (code generation, debugging, documentation)	Covered by existing subscription	In use
Source control	GitHub (free public repository)	Free	In use
Development hardware	Team members' personal machines	—	In use

Deviations from the original budget estimates are negligible. The only direct expenditure was approximately **\$2.50 USD** for the final YOLOv11m training run on cloud GPU (39 epochs, 3 hours of compute on an NVIDIA L40S), which is well below the originally proposed budget. The preliminary CV training runs on Google Colab and the use of the Claude Max plan for development support throughout the project were both covered by pre-existing subscriptions. All other tooling — including the CARLA simulator, the Ultralytics ecosystem, the Python and JavaScript open-source stacks, and the database and AI assistant free tiers — incurred no licensing costs. No hardware purchases were required, as

local development was performed on existing team machines and heavy compute was offloaded to cloud GPU services.

One low-risk future consideration is that the Supabase free tier may prove insufficient for the full tick-level logging volume during final evaluation. If this materializes, the project may evaluate upgrading to Supabase Pro (~\$25/month) or self-hosting Postgres. This is noted as a potential future cost rather than a current blocker.

11. Advisor Feedback and Guidance

A formal advisory meeting was held on 13 May 2026 with Prof. Murat Göksin to review project progress and address ongoing technical challenges. The principal topic of discussion was a difficulty encountered by the computer vision sub-team, which had been struggling to train a detection model capable of reliably classifying emergency vehicles on real-world data. After reviewing the team's approach and preliminary results, Prof. Göksin advised reducing the number of target classes in order to first establish a minimum viable product (MVP) before attempting finer-grained classification. The recommendation prioritizes delivering a working baseline within the available time, after which additional classes can be incrementally reintroduced.

The team has accepted this guidance and is incorporating it into the remaining work plan for the spring term.

12. Conclusion and Next Steps

The project has made strong and consistent progress, with both subsystems having met or exceeded their primary technical objectives ahead of schedule. As of May 2026, the core system — adaptive signal control, SUMO microsimulation, real-time dashboard, backend API, comparative evaluation framework, and the end-to-end computer vision pipeline — is fully implemented and validated. The actuated policy has been quantitatively confirmed to reduce network clearance time by 40–54% over the fixed-time baseline across all three demand profiles, representing a meaningful and reproducible result. The trained YOLOv11 model exceeds the proposal's target accuracy (mAP50 = 0.949 against a target of 0.85), and the analytics layer runs in real time on consumer-grade hardware. Additional features delivered beyond the original proposal scope include emergency vehicle preemption, policy tuning with variant persistence, the CARLA 3D simulation bridge, and an embedded AI assistant.

The remaining work for the spring term is well-defined and achievable within the available time, as summarized below:

Table 8: Remaining Work.

Deliverable	Responsible Sub-Team	Target Date
CV pipeline wired into the simulation tick loop	CV + Backend	End of Week 11
Red-light violation detection and highway entry counting	CV	End of Week 11
Supabase tick-level logging completed	Backend	End of Week 11
Experiment comparison UI finalized	Frontend	Mid Week 12
Dashboard user testing and QA	All	End of Week 12
Final demo environment setup and rehearsal	All	Start of Week 13

Deliverable	Responsible Sub-Team	Target Date
Final report submitted	All	End of Week 13

The team's immediate priority is closing the computer vision integration gap, which is the only major original objective not yet in a production-ready state. All other systems are stable and require only polish and documentation. A potential stretch goal of collision detection from camera input has been identified and will be pursued if the schedule allows after the core remaining work is complete. Overall, the team is confident that the project is on track to deliver a complete, demonstrated, and well-documented AI traffic management system by the end of the spring term.

References

- [1] P. A. Lopez et al., "Microscopic traffic simulation using SUMO," in *Proc. 21st IEEE Int. Conf. Intell. Transp. Syst. (ITSC)*, Maui, HI, USA, Nov. 2018, pp. 2575–2582.
- [2] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proc. 1st Annu. Conf. Robot Learn. (CoRL)*, Mountain View, CA, USA, Nov. 2017, pp. 1–16.
- [3] G. Jocher and J. Qiu, "Ultralytics YOLO11," Ultralytics, 2024. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 779–788.
- [5] Y. Zhang et al., "ByteTrack: Multi-object tracking by associating every detection box," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Tel Aviv, Israel, Oct. 2022, pp. 1–21.
- [6] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Beijing, China, Sep. 2017, pp. 3645–3649.
- [7] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Phoenix, AZ, USA, Sep. 2016, pp. 3464–3468.
- [8] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. 33rd Conf. Neural Inf. Process. Syst. (NeurIPS)*, Vancouver, BC, Canada, Dec. 2019, pp. 8024–8035.
- [9] G. Bradski, "The OpenCV library," *Dr. Dobb's J. Softw. Tools*, vol. 25, no. 11, pp. 120–125, Nov. 2000.
- [10] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Zurich, Switzerland, Sep. 2014, pp. 740–755.
- [11] B. Dwyer, J. Nelson, T. Hansen, et al., "Roboflow (Version 1.0)," Roboflow, 2024. [Online]. Available: <https://roboflow.com>
- [12] S. Ramírez, "FastAPI," 2018. [Online]. Available: <https://fastapi.tiangolo.com>
- [13] Vercel, Inc., "Next.js: The React framework for the web," 2024. [Online]. Available: <https://nextjs.org>
- [14] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2004.
- [15] K. Bernardin and R. Stiefelhagen, "Evaluating multiple object tracking performance: The CLEAR MOT metrics," *EURASIP J. Image Video Process.*, vol. 2008, pp. 1–10, May 2008.
- [16] E. Ristani, F. Solera, R. Zou, R. Cucchiara, and C. Tomasi, "Performance measures and a data set for multi-target, multi-camera tracking," in *Proc. Eur. Conf. Comput. Vis. (ECCV) Workshops*, Amsterdam, The Netherlands, Oct. 2016, pp. 17–35.
- [17] Supabase, Inc., "Supabase: The open-source Firebase alternative," 2024. [Online]. Available: <https://supabase.com>
- [18] Groq, Inc., "Groq API documentation," 2024. [Online]. Available: <https://groq.com>

APPENDIX A: Meeting Minutes and Communication Logs

A.1 Communication Channels

The team relied on the following channels for ongoing coordination throughout the project:

Channel	Purpose	Frequency
WhatsApp / Discord	Day-to-day asynchronous communication, quick clarifications, scheduling	Daily
GitHub (issues, pull requests, code review)	Code coordination, design discussions tied to specific changes	As needed
In-person / video meetings	Weekly synchronization, design reviews, integration planning	Weekly
Email	Formal communication with supervisor and faculty	As needed
Shared documents	Design notes, API specifications, meeting minutes	Continuous

A.2 Recurring Meetings

The team maintained the following recurring meetings to coordinate progress and align on integration:

Meeting	Participants	Frequency	Purpose
Sub-team stand-up (simulation/control)	Alkhozendar, Salamoun, Al-Shobaki	Weekly	Progress updates, blocker resolution, task allocation
Sub-team stand-up (computer vision)	Al-Labani, Dribika, Jayyusi	Weekly	Progress updates, blocker resolution, task allocation
Cross-team integration meeting	All team members	Bi-weekly	Interface alignment, integration planning, demo coordination
Supervisor meeting	All team members + faculty advisor	Bi-weekly	Progress review, feedback, course correction

APPENDIX B: Code Repository / Technical Documentation

The project source code is maintained in two private GitHub repositories, one for each sub-team. Both repositories contain the complete source code, configuration files, scripts, and supporting documentation produced over the course of the project.

Simulation and Control Repository

Repository: <https://github.com/m7md-aiman/TRaffic> (private)

This repository contains the SUMO microsimulation environment, signal control policies, FastAPI backend, Next.js dashboard, the CARLA simulation bridge, and supporting scripts for network generation and race-mode evaluation. Repository structure:

Path	Contents
backend/	FastAPI application — routers, services, WebSocket manager
frontend/	Next.js dashboard — components, hooks, Zustand store
packages/adaptive-policy/	Signal control policies (actuated and fixed-time)
packages/carla-bridge/	CARLA client, traffic manager, camera sensors, junction mapping
packages/cv-pipeline/	Computer vision pipeline integration layer
packages/shared/	Shared Pydantic models, constants, and configuration
packages/sumo-engine/	TraCI client, snapshot extraction, network configuration
scripts/	Network and demand generation, race evaluation runner
supabase/	Database migrations

Computer Vision Repository

Repository: https://github.com/OJayyusiO/dataset_capstone (private)

This repository contains the full source code for the computer vision pipeline, including scenario configurations, dataset capture scripts, model training and evaluation pipelines, and the traffic analytics layer. The repository follows a branch-per-delivery convention, with main as the stable branch and feature branches V2, V2.1, V2.2, and V3.0 each corresponding to a major delivery milestone. A PROJECT_HISTORY.md document within the repository captures every significant design decision and technical challenge encountered during the project, providing a chronological record of the project's evolution.

APPENDIX C: Additional Figures and Data

